Ben Jones individual reflection

Fibonacci heap and Dijkstra's algorithm

Worst case and triggering inputs

For Dijkstra with a Fibonacci heap, the usual bound is O(E + V log V) amortised. A fully reachable dense directed graph forces the algorithm to examine Theta(V squared) edges, even if every weight is positive. Weights that repeatedly improve tentative distances also increase successful priority updates. However, more edges alone do not guarantee more decrease-key operations. Our tested E = cV families are asymptotically sparse, so they do not establish dense-graph behaviour or a crossover.

An individual heap operation can be expensive even when its amortised cost is small. My implementation initially inserts all V vertices as separate roots. The first extract-min can therefore scan a linear number of roots during consolidation. A decrease-key can also trigger a long cascade of cuts. Neither case contradicts the amortised bounds, which account for work across an operation sequence.

Bug and design decision

A concrete issue recorded in the earlier AI-generated Python prototype concerned ownership and cleanup. Circular sibling links could keep removed nodes alive until cyclic garbage collection ran. Correct shortest-path output did not reveal that delayed release. The correction cleared retired links and ownership references and explicitly released the remaining heap structure. A regression test checked release without waiting for cyclic garbage collection, including disconnected cases. This was a prototype issue, not a defect established in the final C++ implementation.

The final C++ design separates ownership from structural links. A vector of unique_ptr objects owns the nodes; parent, child and sibling pointers are non-owning. Extracted handles remain allocated until heap destruction but are marked inactive. This supports invalid-handle checks without accessing freed nodes. Structural operations still need consistent parent links, degrees, marks and a valid minimum pointer.

What I would change

If repeating the work, I would use one Dijkstra body and one output contract with interchangeable heaps. Our implementations differ in initialization, predecessor output, integer types and tie-breaking, so the measured gap cannot be attributed entirely to heap structure. I would also separate allocation and consolidation measurements, retain raw timings and test more seeds and larger, update-heavy inputs. These are proposed changes, not experiments already completed.

The result that surprised me was the performance gap. Binary had lower medians across all nine configurations. Fibonacci still took approximately 3.33 times as long at 4,000 vertices and E/V = 64. This comparison shows why complexity analysis and complete-program measurement both matter.

How I interpret the result now

Earlier in the project, I asked whether Fibonacci was expected to be much better. The distinction I take from the comparison is between an asymptotic operation bound and complete-program runtime. Constant amortised decrease-key does not mean every call takes constant time or that Dijkstra necessarily finishes faster. A decrease can cause several cuts. Marking after the first child loss and cutting after the second preserves the structural bound, while amortised analysis accounts for work across operations. Initialization, allocation, output work and workload size still affect measured time. Our binary implementation can therefore win without contradicting Fibonacci's theoretical advantage.